



Universidade do Minho
Licenciatura em Engenharia Informática

Sistemas Operativos

Ano Lectivo de 2025/2026

TPSO27

Rui Cruz (a104355)

João Pedro (a104533)

SO

Índice

Resumo.....	3
1. Introducao.....	3
2. Visao geral da arquitetura.....	3
3. Organizacao do codigo.....	4
4. Fluxo de funcionamento do sistema.....	5
4.1 Submissao e execucao de um comando.....	5
4.2 Consulta do estado global.....	6
4.3 Encerramento controlado.....	6
5. Detalhes de implementacao.....	7
5.1 Protocolo de mensagens e dados partilhados.....	7
5.2 Escalonamento e estado interno do controller.....	7
5.3 Execucao de pipelines e redirecionamentos.....	9
6. Registo persistente e observabilidade.....	9
7. Testes e avaliacao experimental.....	9
8. Conclusao.....	10

Resumo

Este trabalho apresenta a implementação de um serviço de execução de comandos em Linux baseado numa arquitetura cliente-servidor composta por um processo central, designado `controller`, e por múltiplos clientes `runner`. A solução permite submeter comandos, consultar o estado global do sistema e encerrar o serviço de forma controlada, mantendo sempre registo persistente da atividade executada. A implementação foi desenvolvida em C com recurso direto a chamadas ao sistema, utilizando FIFOs para comunicação entre processos, `fork` e `execvp` para execução de comandos, `pipe` e `dup2` para suporte a pipelines e redirecionamentos, e `gettimeofday` para medição temporal. Para além da funcionalidade principal, o projeto inclui scripts de teste e de benchmark que permitem validar o comportamento do sistema e observar o impacto das políticas de escalonamento e dos níveis de paralelismo.

1. Introdução

O objetivo central do projeto foi construir um pequeno ambiente de execução coordenada de comandos no qual diferentes utilizadores podem submeter trabalho concorrente a um processo responsável por ordenar, autorizar e acompanhar a sua execução. Em vez de cada cliente executar imediatamente o comando pedido, o modelo adotado introduz uma fase explícita de arbitragem: o `runner` comunica primeiro com o `controller`, aguarda autorização e só depois inicia a execução local do comando. Esta separação de responsabilidades torna visíveis vários conceitos nucleares de Sistemas Operativos, nomeadamente comunicação entre processos, gestão de concorrência, sincronização, redirecionamento de descritores e observabilidade do sistema.

A implementação desenvolvida privilegia uma arquitetura simples, mas robusta. O `controller` concentra a lógica de escalonamento e o estado global do sistema, enquanto o `runner` se ocupa da interação com o utilizador e da execução efetiva do comando autorizado. Esta divisão permite que a política de decisão fique isolada num ponto único, evitando ambiguidades quando vários clientes tentam executar comandos em simultâneo.

2. Visão geral da arquitetura

A arquitetura é composta por dois binários principais. O `controller` funciona como servidor do sistema: recebe pedidos, mantém a fila de comandos pendentes, decide quando cada comando pode arrancar, disponibiliza listagens do estado atual e encerra apenas depois de todo o trabalho em curso ter sido drenado. O `runner` funciona como cliente: interpreta o modo de utilização indicado na linha de comandos, cria um canal de resposta privado, envia pedidos ao `controller` e, no modo de execução, arranca o comando apenas quando recebe uma confirmação afirmativa.

A comunicação entre processos assenta em dois tipos de FIFO. Existe uma FIFO principal do controller, partilhada por todos os clientes, para a receção de pedidos. Para cada operação, o runner cria ainda uma FIFO temporária privada, indicada no pedido, por onde recebe a resposta específica dessa interação. Esta abordagem evita a mistura de respostas entre clientes concorrentes e permite manter o protocolo de comunicação simples, determinístico e facilmente observável.

No interior do controller, o estado é representado por duas listas principais: uma lista de comandos agendados e uma lista de comandos em execução. Sempre que chega um novo pedido de execução, o comando entra na fila de espera com um carimbo temporal de submissão. Quando existe capacidade livre de acordo com o paralelismo configurado, o controller escolhe a próxima entrada de acordo com a política ativa, envia uma mensagem de autorização ao respetivo runner e move o comando para a lista de execução, registando o instante de despacho. Quando o runner termina, envia uma notificação final, permitindo ao controller remover o comando da lista ativa e escrever uma linha persistente de log com os tempos recolhidos.

3. Organização do código

O projeto foi estruturado em módulos pequenos e com responsabilidades bem definidas. Esta organização torna o código mais fácil de testar, manter e explicar.

Ficheiro	Funcao no projeto
Makefile	Define a compilação dos dois executáveis, a criação da árvore obj/ e bin/ e a limpeza dos artefactos gerados.
include/common.h	Centraliza constantes, estruturas de mensagens e protótipos usados pelos dois binários. E o ponto comum do protocolo de comunicação.
include/runner_exec.h	Expõe a interface do módulo responsável por interpretar e executar a linha de comandos do utilizador.
src/common.c	Implementa utilitários partilhados: escrita e leitura completas em descritores, construção segura de caminhos e strings, gestao dos caminhos do runtime e geração de identificadores de comando.
src/controller.c	Contém o ciclo principal do servidor, a gestão das filas de comandos, a política de escalonamento, o tratamento dos pedidos recebidos e o registo persistente em log.

src/runner.c	Implementa a interface cliente, a criação das FIFOs privadas de resposta, o envio de pedidos ao controller e os modos -e, -c e -s.
src/runner_exec.c	Implementa o parser da linha de comandos e a execução real de pipelines e redirecionamentos com fork, pipe, dup2 e execvp.
scripts/test_basic.sh	Executa um fluxo funcional mínimo para validar compilação, submissão e encerramento do sistema.
scripts/test_parallel.sh	Exercita o controller com vários runner em paralelo, avaliando se o limite de concorrência é respeitado.
scripts/test_orphan_runner.sh	Simula a morte de um runner durante a execução para confirmar que o controller limpa o estado órfão e não bloqueia o encerramento.
scripts/test_requirements.sh	Reúne uma bateria de testes funcionais que cobre execução, consulta, redirecionamentos, shutdown e concorrência.
scripts/benchmark_policies.sh	Automatiza experiências comparativas entre políticas de escalonamento, recolhendo tempos médios por utilizador.

A separação entre `runner.c` e `runner_exec.c` merece destaque. O primeiro módulo trata da comunicação com o `controller` e da experiência do utilizador; o segundo trata exclusivamente da interpretação e execução do comando autorizado. Esta divisão evita que a lógica de IPC fique misturada com a lógica de processos filhos e redirecionamentos, o que simplifica tanto a implementação como a análise do código.

4. Fluxo de funcionamento do sistema

4.1 Submissão e execução de um comando

No modo de execução, o `runner` recebe o identificador do utilizador e a linha de comandos a executar. O primeiro passo consiste em reconstruir essa linha de forma segura, preservando os argumentos fornecidos pelo utilizador. Em seguida, o `runner` cria uma FIFO temporária privada que servirá de canal de resposta e construirá uma estrutura `RequestMessage` com o tipo de pedido, o identificador do comando, o PID do próprio `runner`, o utilizador e a string do comando.

Depois de enviar o pedido para a FIFO principal do `controller`, o `runner` bloqueia na leitura da FIFO privada, aguardando autorização. Este comportamento é importante porque impede que o cliente avance antes de o sistema global decidir se o comando pode ou não iniciar. Quando

existe vaga, o controller responde com `RESP_ACK_EXEC`, momento a partir do qual o runner imprime a mensagem de início, executa o comando e, no final, envia um `REQ_FINISH` com o código de saída obtido.

Do lado do controller, a submissão segue um percurso complementar. Ao receber `REQ_EXEC`, o servidor valida se o sistema está ou não em processo de encerramento e verifica se o `command_id` proposto não entra em conflito com comandos já ativos ou pendentes. Se a submissão for válida, cria uma entrada interna com todos os metadados do pedido e coloca-a na fila de escalonamento. Mais tarde, quando o escalonador decide despachar esse comando, o controller envia a resposta afirmativa para a FIFO privada do runner, regista o instante de dispatch e move a entrada para a lista de comandos em execução.

4.2 Consulta do estado global

No modo `-c`, o runner não executa qualquer comando. Em vez disso, cria uma FIFO privada e envia ao controller um pedido de listagem. O controller responde com uma sequência de mensagens `RESP_LIST`, onde concatena o cabeçalho da seção `Executing`, a lista de comandos em execução, o cabeçalho `Scheduled` e a lista de comandos em espera. A listagem apresentada ao utilizador não é produzida localmente pelo cliente; ela é sempre derivada do estado mantido pelo controller, o que garante consistência com a realidade global do sistema naquele instante.

Um aspeto relevante desta implementação é que o controller trata o pedido de listagem num processo filho criado com `fork`. Esta opção evita que a preparação e envio da resposta possam atrasar o ciclo principal do servidor, sobretudo quando existirem vários pedidos concorrentes ou quando a FIFO de resposta demorar a ser drenada pelo cliente.

4.3 Encerramento controlado

No modo `-s`, o runner envia um pedido de shutdown e fica a aguardar uma confirmação final. O controller não termina imediatamente ao receber esse pedido. Em vez disso, entra num estado de encerramento pendente, deixa de aceitar novas execuções, continua a acompanhar os comandos já ativos e só sai quando deixa de existir trabalho em execução e em espera. Este desenho garante um encerramento gracioso, sem perda de informação nem abandono de comandos a meio.

Para suportar vários pedidos de encerramento, o controller guarda uma lista de `watchers`, isto é, uma lista das FIFOs privadas dos clientes que pediram o shutdown. Quando o sistema fica efetivamente drenado, envia uma notificação `RESP_SHUTDOWN_DONE` para todos esses canais, libertando assim os runner que estavam bloqueados à espera do fim do serviço.

5. Detalhes de implementação

5.1 Protocolo de mensagens e dados compartilhados

O protocolo de comunicação foi definido em `include/common.h` através de duas estruturas de tamanho fixo: `RequestMessage` e `ResponseMessage`. Esta escolha simplifica bastante a implementação, porque evita parsing textual, reduz ambiguidades e permite escrever e ler blocos de memória de tamanho conhecido. Os tipos de pedido distinguem submissão, conclusão, listagem e encerramento, enquanto os tipos de resposta cobrem autorização, rejeição, listagem e confirmação de shutdown.

O módulo `src/common.c` agrega as funções auxiliares usadas por ambos os binários. Entre elas encontram-se `write_full` e `read_full`, que encapsulam leituras e escritas robustas sobre descritores, tolerando interrupções por sinais; `safe_snprintf`, usada na construção segura de caminhos e mensagens; `build_command_line`, que recompõe a linha de comandos a partir dos argumentos do runner; e `generate_command_id`, que combina tempo, PID e uma sequência local para produzir identificadores com baixa probabilidade de colisão.

5.2 Escalonamento e estado interno do controller

O controller suporta duas políticas de escalonamento: FIFO é uma variante simples de round-robin por utilizador. Em FIFO, a fila é percorrida por ordem de chegada. Em RR, o servidor tenta evitar despachar repetidamente comandos do mesmo utilizador quando existam alternativas na fila. Para esse efeito, o estado interno guarda o identificador do último utilizador despachado e procura, sempre que possível, a próxima entrada pertencente a um utilizador diferente.

A política RR implementada não reordena comandos em execução nem faz preempção; trata-se de uma estratégia de escolha do próximo pedido a despachar. Esta opção manteve a implementação compatível com o modelo do trabalho, reduzindo complexidade e continuando a introduzir um critério de justiça observável nos testes comparativos.

Outro aspeto importante do controller é a limpeza de estado órfão. O servidor verifica periodicamente, com `kill(pid, 0)`, se os runner associados aos pedidos ainda existem. Se um cliente morrer antes de terminar o seu protocolo, a entrada correspondente é removida da fila ou da lista de execução e é gerada uma linha de log apropriada. Esta limpeza impede que comandos sem cliente associado bloqueiem vagas de paralelismo ou impeçam o encerramento do sistema.

5.3 Execução de pipelines e redirecionamentos

A execução efetiva dos comandos foi isolada em `src/runner_exec.c`. O módulo começa por tokenizar a linha de comandos, reconhecendo palavras, pipes e os operadores de redirecionamento `<`, `>`, `2>` e `|`. De seguida, constroi uma representação interna composta por estágios, em que cada estágio corresponde a um comando de uma pipeline e pode ter ficheiros de entrada, saída e erro associados.

Depois do parsing, a execução é realizada com a criação dos pipes necessários e de um processo filho por estágio. Cada filho herda os descritores adequados, aplica `dup2` para ligar `stdin`, `stdout` e `stderr` aos pipes ou ficheiros corretos, fecha os descritores que deixaram de ser necessários e chama `execvp` para substituir a imagem do processo. No processo pai, a espera pelos filhos é feita com `waitpid`, sendo o código de saída final devolvido ao runner para posterior comunicação ao controller.

Esta abordagem tem duas vantagens principais. Em primeiro lugar, executa os comandos sem recorrer a `system()` nem a uma shell externa, mantendo a lógica do trabalho inteiramente dentro das primitivas estudadas na unidade curricular. Em segundo lugar, permite controlar explicitamente a criação e fecho de descritores, tornando o comportamento da pipeline observável e previsível.

6. Registo persistente e observabilidade

Sempre que um comando termina, o controller escreve uma linha em `tmp/controller.log` com o identificador do utilizador, o identificador do comando, os instantes de submissão, despacho e conclusão, os tempos de espera, execução e duração total, o código de saída e a política de escalonamento em uso. Este formato textual foi escolhido por ser simples de inspecionar, fácil de processar em scripts e suficientemente rico para suportar análise experimental.

A existência deste log transforma o sistema numa plataforma observável. Com base nele, é possível perceber se um comando passou demasiado tempo em espera, confirmar a ordem real de despacho, relacionar a política ativa com os tempos observados e detetar anomalias como comandos órfãos ou terminações com erro. Os scripts de benchmark aproveitam precisamente esta característica para calcular médias por utilizador e por política.

7. Testes e avaliação experimental

A validação do projeto foi feita em dois níveis complementares. O primeiro corresponde a testes funcionais dirigidos, pensados para confirmar que cada operação principal do sistema se

comporta como esperado. O segundo corresponde a testes de carga e benchmark, usados para observar o comportamento do escalonador em cenários com vários clientes concorrentes.

A compilação do projeto foi validada com `make clean && make all`, garantindo que os dois binários são gerados corretamente a partir da árvore de código fonte. Sobre essa base, os scripts em `scripts/` automatizam os cenários mais importantes. `test_basic.sh` cobre o arranque do servidor, a submissão simples de um comando e o encerramento do serviço. `test_parallel.sh` submete vários comandos concorrentes para confirmar que o limite de paralelismo imposto ao controller é respeitado. `test_orphan_runner.sh` introduz uma falha artificial num cliente para verificar que o estado interno é limpo e que o sistema continua a ser encerrado. Finalmente, `test_requirements.sh` agrega um conjunto mais alargado de verificações sobre execução, listagem, redirecionamentos, pipelines e shutdown.

Para a avaliação comparativa das políticas, foi utilizado o script `benchmark_policies.sh`, que cria várias execuções concorrentes distribuídas por diferentes utilizadores e calcula tempos médios de espera e duração total a partir do log. Os resultados recolhidos para uma execução com doze jobs, três utilizadores e paralelismo dois encontram-se resumidos na tabela seguinte.

Policy	User	Count	Avg Wait (ms)	Avg Total (ms)
fifo	1	12	4071.83	5843.83
fifo	2	12	3127.00	4787.75
fifo	3	12	5273.08	6937.08
rr	1	12	3459.92	5254.08
rr	2	12	4247.42	5898.17
rr	3	12	4539.50	6180.00

A leitura destes valores deve ser feita com algum cuidado. Em cargas heterogéneas, uma política round-robin simples não garante distribuição perfeita dos tempos médios, porque os comandos não têm todos a mesma duração e o modelo não é preemptivo. Ainda assim, os resultados mostram que a política influencia de forma clara a experiência de espera dos utilizadores e que a instrumentação adotada permite discutir essa diferença com dados concretos, e não apenas com intuição.

Em termos qualitativos, os testes também mostraram que o sistema lida corretamente com redirecionamentos e pipelines, que o shutdown só ocorre depois do esvaziamento do trabalho pendente e que a limpeza de runner órfãos evita bloqueios no controller. Esta componente

de robustez foi particularmente importante para garantir que o serviço se mantém coerente mesmo perante terminações anormais dos clientes.

8. Conclusao

O projeto resultou numa implementação coesa de um ambiente multi-runner, construída diretamente sobre as primitivas de Sistemas Operativos estudadas na unidade curricular. A arquitetura adotada separa com clareza as responsabilidades de coordenação e de execução, o que simplifica o raciocínio sobre concorrência e torna o comportamento do sistema mais fácil de testar e explicar. A organização modular do código, em especial a divisão entre lógica de comunicação, lógica de escalonamento e lógica de execução de comandos, contribuiu para uma solução mais legível.

Do ponto de vista funcional, o sistema suporta submissão de comandos, consulta do estado, encerramento controlado, logging persistente e execução de pipelines com redirecionamentos. Do ponto de vista experimental, os scripts disponibilizados permitem repetir testes, comparar políticas de escalonamento e observar o impacto do paralelismo em cenários com vários utilizadores. Em conjunto, estes aspetos fazem do trabalho não apenas uma implementação correta do serviço pedido, mas também uma demonstração clara de conceitos fundamentais de comunicação entre processos, gestão de descritores, sincronização e observabilidade em sistemas Unix.